TECHNICAL REPORT · NORMALIZATION TECHNIQUES · DEEP LEARNING

# Layer *Normalization*

A rigorous exploration of Layer Normalization — the stabilization mechanism that tames the chaotic activation distributions inside deep transformer networks, enabling training at depth and scale.

• ACTIVE RESEARCH AREA | TRANSFORMERS | TRAINING STABILITY | NORMALIZATION | DEEP LEARNING


## Uditya Narayan Tiwari

// Technical Author · AI Researcher

Portfolio | GitHub | LinkedIn | Knowledge Base


// CONTENTS

// 01

# The Problem: *Internal Covariate Shift*

As a neural network trains, the distribution of activations flowing through each layer changes continuously — every weight update at layer *L* shifts the distribution that layer *L+1* must process. This phenomenon, termed **Internal Covariate Shift** by Ioffe & Szegedy (2015), forces deeper layers to perpetually adapt to a moving target.

Without normalization, deep networks suffer from **vanishing gradients** (activations collapse toward zero), **exploding gradients** (activations blow up), and **slow convergence** requiring meticulous learning rate tuning. Layer Normalization (Ba et al., 2016) was introduced as an alternative to Batch Normalization that works along the *feature dimension* — making it independent of batch size and compatible with recurrent and transformer architectures.

> *"Unlike batch normalization, layer normalization directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer." — Ba et al., 2016*

⚡ **Why transformers need it:** Attention and feed-forward layers can produce activations of wildly varying scale. Without normalization, residual streams in deep models (e.g. 96-layer GPT-3) would quickly lose signal. LayerNorm is what makes extreme depth trainable.

// 02

# The LayerNorm *Formula*

Given an input vector `x ∈ ℝ^H` (H = hidden dimension), Layer Normalization computes statistics *across the feature dimension* for that single token/sample, then normalizes and rescales.

CORE DEFINITION

$$\text{LayerNorm}(x) = \gamma \odot \hat{x} + \beta$$

where the normalized activations $\hat{x}$ are:

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$$

Statistics computed over the feature dimension H:

$$\mu = (1/H) \cdot \Sigma_i \, x_i \qquad \text{(mean)}$$

$$\sigma^2 = (1/H) \cdot \Sigma_i \, (x_i - \mu)^2 \qquad \text{(variance)}$$

Learnable parameters:

- $\gamma \in \mathbb{R}^H$ — scale (gain), initialized to 1
- $\beta \in \mathbb{R}^H$ — shift (bias), initialized to 0
- $\varepsilon$ — numerical stability constant (e.g. 1e-5)

EXPANDED ELEMENT-WISE FORM

For each feature dimension $i \in \{1, \ldots, H\}$:

$$y_i = \gamma_i \cdot ((x_i - \mu) / \sqrt{(\sigma^2 + \varepsilon)}) + \beta_i$$

Notice: $\mu$ and $\sigma^2$ are SCALARS shared across ALL features of one token. BatchNorm uses scalars shared across all tokens of one feature. This is the fundamental difference.

// 03

# Step-by-Step Computation

Walk through the full forward pass of LayerNorm on a single token vector `x ∈ ℝ^H`:

01

## Compute the Mean μ

Average across all H feature dimensions of the token: `μ = (1/H) · Σᵢ xᵢ`. This is a single scalar that captures the center of the distribution for this token's representation. Note: this happens independently per token — no information crosses between tokens.

02

## Compute the Variance σ²

Compute the average squared deviation from the mean: `σ² = (1/H) · Σᵢ (xᵢ - μ)²`. This scalar measures the spread of activations. Large variance means the activation range is wide and needs to be compressed; small variance means it is already tight.

03

## Subtract Mean & Normalize

Center and scale: `x̂ᵢ = (xᵢ - μ) / √(σ² + ε)`. After this step, the vector has mean ≈ 0 and standard deviation ≈ 1. The `ε` (epsilon) term prevents division by zero when σ² ≈ 0. Typical values: `ε = 1e-5`.

04

## Apply Learned Affine Transform

Rescale and shift: `yᵢ = γᵢ · x̂ᵢ + βᵢ`. Pure normalization loses expressive power — the network can no longer represent the identity function. The learned γ (scale) and β (bias) parameters restore this capacity, allowing the model to recover any distribution it finds optimal during training.

05

## Backward Pass (Gradient Flow)

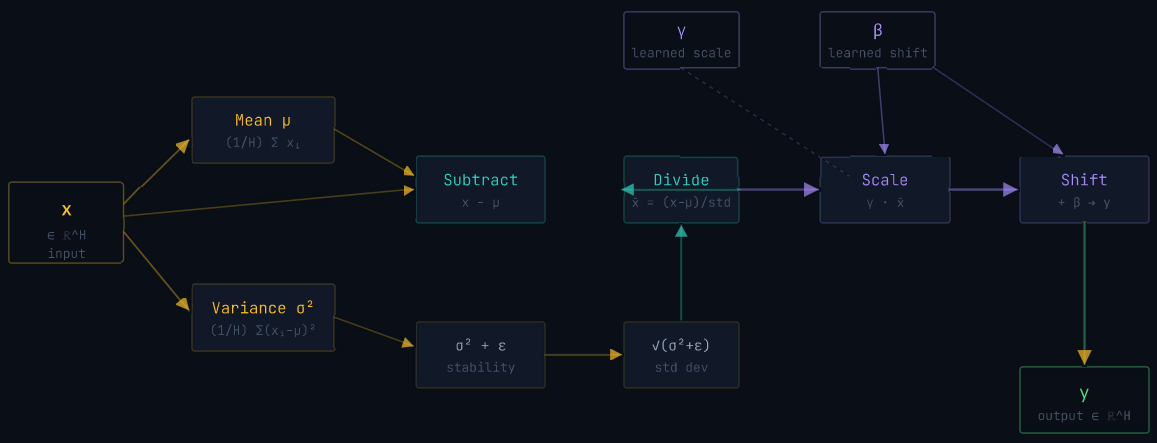
Gradients flow through γ and β trivially (linear). The normalization step involves a non-trivial Jacobian, but the whitened representation ensures gradients remain in a well-conditioned

space — preventing vanishing/exploding gradients through layer depth.

// 04

# Anatomy of a *LayerNorm*

// LAYERNORM FORWARD PASS — VISUAL DIAGRAM




🔍 **Critical observation:** The entire normalization (μ, σ²) is computed from *one sample's features*. This means LayerNorm has zero dependency on batch size — it works identically whether the batch has 1 or 1,000,000 samples. This is what makes it ideal for autoregressive inference.

// 05

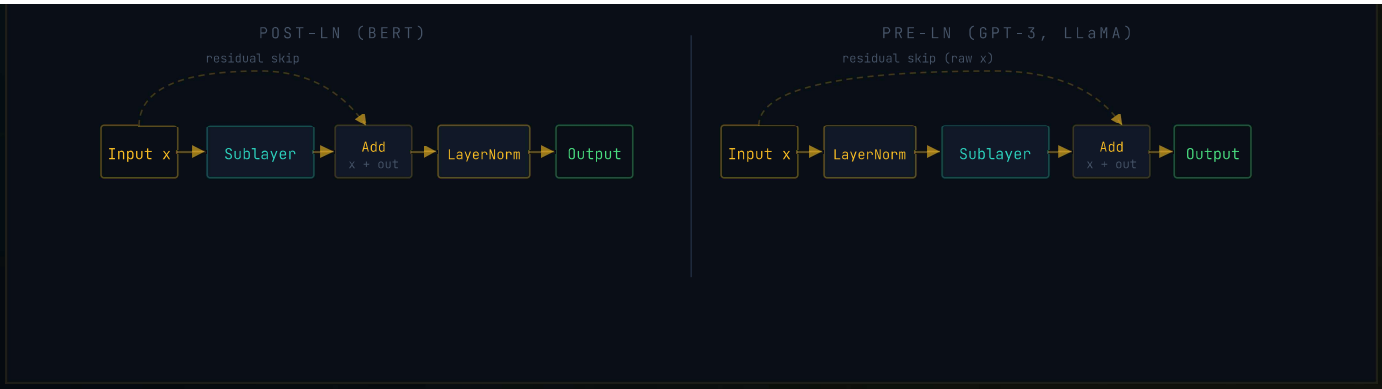
# Pre-Norm vs *Post-Norm* Placement

Where LayerNorm is placed relative to the sub-layers (attention, FFN) dramatically affects training dynamics. This choice has evolved significantly across model generations.

| PROPERTY | POST-LN (ORIGINAL) | PRE-LN (MODERN) | RMS PRE-LN (LATEST) |
|---|---|---|---|
| Formula | `LN(x + Sublayer(x))` | `x + Sublayer(LN(x))` | `x + Sublayer(RMS(x))` |
| Used In | Original Transformer (2017), BERT | GPT-2, GPT-3, PaLM | LLaMA, Mistral, Gemma |
| Training Stability | ~ Requires warmup | ✓ More stable | ✓ Most stable |
| Final Performance | ✓ Slightly better | ~ Competitive | ✓ Competitive |
| Gradient Scale | ✗ Grows with depth | ✓ Bounded | ✓ Bounded |
| Residual Stream | Normalized after residual | Raw residual preserved | Raw residual preserved |

**Pre-LN insight:** With Pre-LN, the residual stream `x` flows unchanged through the addition — its scale grows slowly and predictably with depth. Gradients can flow directly through the residual path without passing through normalization, dramatically improving stability for very deep models.

```
// PRE-LN VS POST-LN BLOCK STRUCTURE
```




// 06

# Variants & Evolutions

The normalization landscape has evolved significantly, with each variant targeting specific inefficiencies or use cases.

### LayerNorm (2016)

| | |
|---|---|
| Paper | Ba et al. |
| Params | 2H (γ, β) |
| Stats | mean + variance |
| Used in | BERT, GPT-2 |

### RMSNorm (2019)

| | |
|---|---|
| Paper | Zhang & Sennrich |
| Params | H (γ only) |
| Stats | RMS only (no mean) |
| Used in | LLaMA, Mistral |

### GroupNorm (2018)

| | |
|---|---|
| Paper | Wu & He |
| Params | 2H (γ, β) |
| Stats | per feature group |
| Used in | Vision models |

### DeepNorm (2022)

| | |
|---|---|
| Paper | Wang et al. |
| Params | 2H + α scalar |
| Stats | scaled residual |
| Used in | Very deep models |

## RMSNorm — The Modern Successor

RMSNorm (Root Mean Square Normalization) drops the mean-centering step entirely, computing only the root mean square of activations. Zhang & Sennrich (2019) showed this achieves comparable performance with ~7% faster training speed.

RMSNORM FORMULA

$$\text{RMSNorm}(x) = \gamma \odot x \,/\, \text{RMS}(x)$$

where:

$$\text{RMS}(x) = \sqrt{(1/H) \cdot \Sigma_i \, x_i^2 \;+\; \varepsilon}$$

Key differences from LayerNorm:

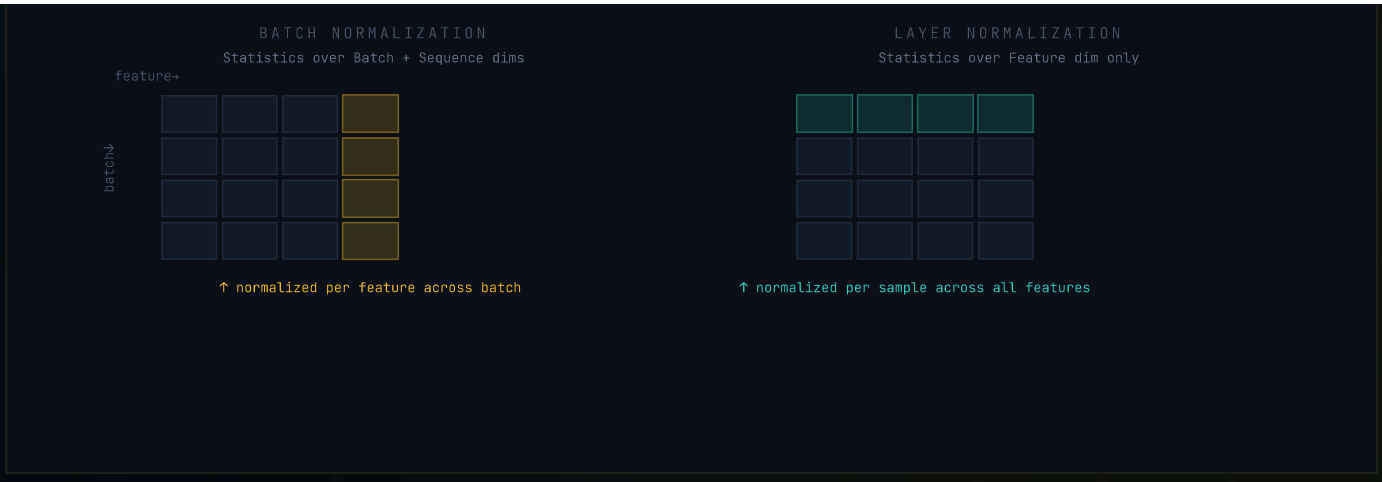
- ✗ No mean subtraction (μ removed)
- ✗ No bias parameter β (half the parameters)
- ✓ Faster: one fewer pass over data
- ✓ Hypothesis: re-centering is less important than re-scaling

// 07

## LayerNorm vs BatchNorm

The key distinction between these two dominant normalization strategies lies in *which dimensions* the statistics are computed across.

// NORMALIZATION AXIS VISUALIZATION (BATCH × TOKENS × FEATURES)




| PROPERTY | BATCHNORM | LAYERNORM |
|---|---|---|
| Statistics computed over | Batch × Sequence dimensions | Feature dimension only |
| Batch size dependency | ✗ Requires large batches | ✓ Batch-size independent |
| Inference vs Training | ~ Different (uses running stats) | ✓ Identical behavior |
| Sequences of varying length | ✗ Requires padding adjustments | ✓ Handles naturally |
| Autoregressive decoding | ✗ Batch of 1 is unstable | ✓ Works perfectly |
| CV/image tasks | ✓ Preferred (large batches) | ~ Works, less optimal |
| Primary use case | CNNs, image classification | Transformers, NLP, multimodal |

// 08

## Role in the *Transformer Block*

In the standard transformer, LayerNorm appears twice per block — once before (or after) the Multi-Head Attention sub-layer, and once before (or after) the Feed-Forward Network. Each

placement serves the same purpose but on different activation distributions.

| LAYERNORMS PER BLOCK | PARAMETERS PER LN | TOTAL LN PARAMS (GPT-3) |
|---|---|---|
| 2× | 2H | 96×2 |
| One per sub-layer | γ and β vectors | 192 LayerNorm layers |

| COMPUTE OVERHEAD |
|---|
| <1% |
| vs. attention + FFN |

🧠 **The big picture:** LayerNorm is the quiet workhorse of the transformer. While attention and FFN layers do the heavy representational lifting, LayerNorm ensures the activations flowing between them stay in a stable, well-conditioned regime — making it the difference between a model that trains and one that diverges.

🔬 **Research frontier:** Recent work (e.g. Noci et al. 2022) shows that removing LayerNorm from transformers entirely is possible with careful initialization (the "NoNorm" hypothesis). However, for practical training of large models, LayerNorm (or RMSNorm) remains the standard choice due to its robustness and negligible cost.

## 📚 Further Resources & Notes

All implementations, derivations, and extended notes by the author are available at:

https://udityaknowledgebase.netlify.app/

Key References: Ba et al. (2016) "Layer Normalization" · Zhang & Sennrich (2019) "Root Mean Square Layer Normalization" · Ioffe & Szegedy (2015) "Batch Normalization" · Wang et al. (2022) "DeepNet" · Xiong et al. (2020) "On Layer Normalization in the Transformer Architecture"

---


## Uditya Narayan Tiwari

// AI Researcher · Technical Author


Layer Normalization — Technical Report


→ Portfolio

→ GitHub

→ LinkedIn

→ Knowledge Base